

ZeroFlush: A CSD-Native LSM Store with In-Storage Write Path Organization

ZeroFlush

1. Background

- LSM write path: WAL → MemTable → Flush → Compaction
- WAL & flush persist the same data → 2× pre-L1 NAND writes
- CSD brings compute to data (FPGA + device CPU)

2. Motivation

- Flush re-writes already-persisted bytes → 2× pre-L1 WA
- Time-ordered MemTable → overlapping L0 → read fan-in
- Compaction lag → foreground write stalls

3. Gap

- Prior work mitigates (tuning, Titan, offload, ADOC); none eliminates the flush pass

4. Observation

- Overlap is decided at admission, not compaction
- Ordered runs need not be materialized — the premise is obsolete

5. Solution

- Persist Once, Organize In-Place (pre-L1 scope)

6. Challenge & goals

- C1 ordered input / C2 readable data / C3

visibility+recovery; goals G1–G3

7. Design

- D1 partition · D2 fold · D3 slim index · D4 CSD merge · D5 adaptive scan

8. Evaluation

- Claim-driven matrix + ablation + crash injection

1. Background: The Four-Step LSM Write Path

The conventional LSM write path has four steps:

- 1) **WAL append: durable log**
- 2) MemTable: in-memory order and version lookup
- 3) Flush: materialize L0 blocks, index, and Bloom filter
- 4) Compaction: merge L0 into L1+ and persist the next-level run

Before L1, WAL and flush materialize the same update.

ZeroFlush eliminates only step 3: partitioned WAL feeds step 4 directly.

Step 4 and all subsequent compaction writes remain.

Scope: "2×" counts pre-L1 NAND passes, not end-to-end write amplification (typically 10–30×); both are reported in Evaluation.

2. Motivation: Why the Flush Must Go

The write-path cost is structural - not a tuning problem. Three pain points make the flush pass the prime target:

Pain 1: Flush re-writes the same bytes

WAL already persisted the data; flush writes it again just to sort + encode.
Cost: 2x NAND write amplification to L1.

Pain 2: Overlapping L0 hurts reads

MemTable fills by time -> flush emits overlapping L0 files: point queries check ALL L0; L0->L1 compaction merges many.

Pain 3: Write stalls block foreground

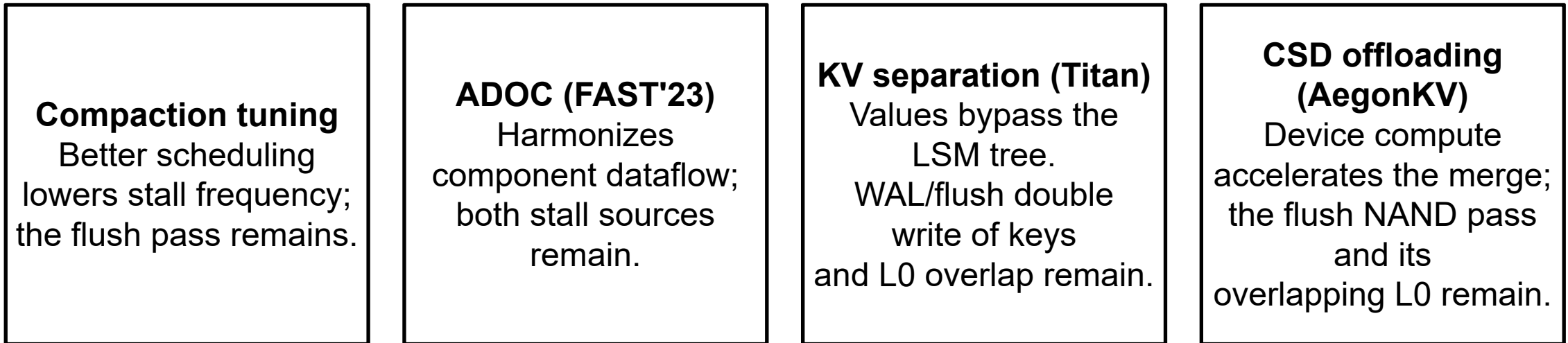
When compaction falls behind, foreground writes stall.
ADOC (FAST'23): stalls trace to flush-limited and compaction-limited dataflow mismatch.

Every pain has been mitigated; none has been eliminated (next: the gap).

=> To remove the cost, we must revisit the premises the write path rests on.

3. Gap: Mitigation Is Not Elimination

Four families of prior work reduce the symptoms; all keep the flush pass and the premises behind it.



Positioning: ADOC harmonizes the dataflow; ZeroFlush eliminates two of its stall sources by construction.

No block-interface LSM engine eliminates the flush pass: prior work mitigates it (KV separation, tuning), relocates it into the device (KV-SSDs, CSD offload), or changes the persistence medium (PM).

4. Observation: Where the Cost Actually Comes From

Obs. 1: Overlap is decided at admission

MemTable fills by time, not key range, so flush emits overlapping L0. Prior work mitigates at compaction; none at admission.

Obs. 2: Materialization, not ordering, is obsolete

The path assumes ordered runs must be materialized as NAND files. A DRAM index can guide device reads to synthesize a sorted stream — no materialized L0.

Obs. 3: Device compute makes this actionable

CSD pairs NAND with FPGA + CPU. Organizing data at persistence time is feasible — if the device sustains the merge rate (checked by microbenchmark).

Insight: if the device organizes data at persistence time, the flush pass - and the overlap it creates - can be eliminated entirely.

Feasibility is checked by microbenchmark; then three barriers stand in the way (challenge).

5. Solution: Persist Once, Organize In-Place

Remove the separate WAL-to-L0 materialization.

A confirmed update is durable in a partitioned WAL record; the same record later feeds the L1 merge directly.

Scope: the claim removes the flush write (pre-L1), not later compaction writes.

CSD is not an accelerator. It enables a new write path.

Traditional: Host organizes -> SSD stores

ZeroFlush: Host writes -> CSD organizes in place

6. Challenge & Design Goals

Challenges are derived, not asserted: removing flush removes three services it used to provide.

(1) Ordered input

WAL records arrival order, whereas compaction needs a key-sorted input stream.

(2) Readable data

L0 SSTables provide block indexes, Bloom filters, and ordered scans.

(3) Concurrent visibility and recovery

The device reorganizes historical WAL while the host keeps writing. Publication and crash recovery must preserve LSM version visibility.

Design goals (make the mapping auditable):

- G1 no second NAND write on the foreground path.
- G2 read regression bounded by ϵ (point + range).
- G3 crash-recovery semantics equivalent to a WAL-based LSM.

ZeroFlush: System Overview

Host side (redesigned):

- MemTable SLIMMED, not removed: skiplist index + value cache
- WAL written in partitions, dynamically aligned to L1
- Each sealed WAL partition feeds compaction directly (no flush)
- Range scans: adaptive policy over partitioned WAL (D5)

SmartSSD side (new):

- CSD Merge Compaction Engine on FPGA (P2P, no PCIe)
- Reads WAL partition by index order -> sorted stream, no sort

Mechanism -> challenge/goal mapping:

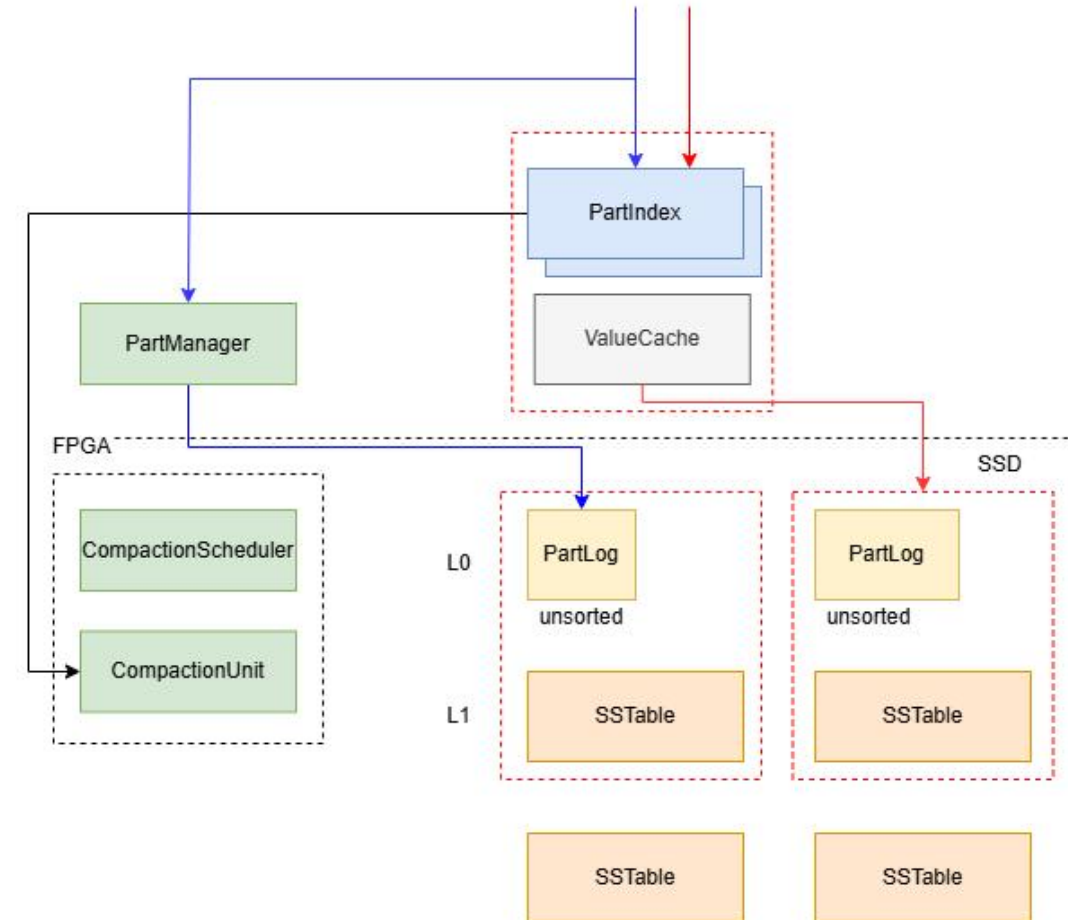
D1 WAL partitioning -> Obs.1 payoff at admission; C2 bounded units

D2 folded write -> Pain 1; G1 (one pre-L1 NAND pass)

D3 slim MemTable -> C1 sorted stream; C2 point reads (G2)

D4 CSD merge compaction -> C1, C3 (fence + validation); G3

D5 adaptive range scan -> C2 range reads (G2)

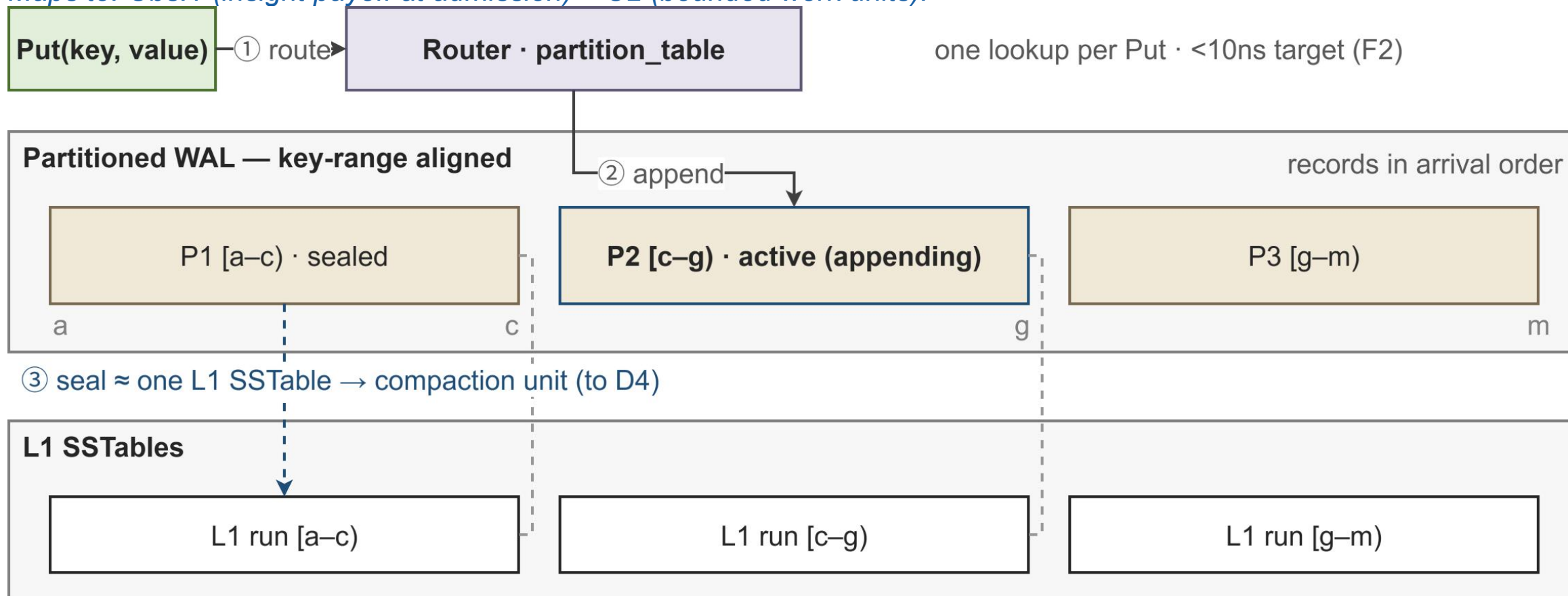


7. Design (1): WAL Partitioning with Dynamic L1 Alignment

Dynamic, admission-time L1 alignment:

boundary minimizes $\text{cost}(p) = \text{overlap_bytes}(p, L1) + \lambda \cdot \text{size}(p)$; sealed unit $\approx$ one L1 SSTable — exactly the compaction unit.

Maps to: Obs.1 (insight payoff at admission) + C2 (bounded work units).



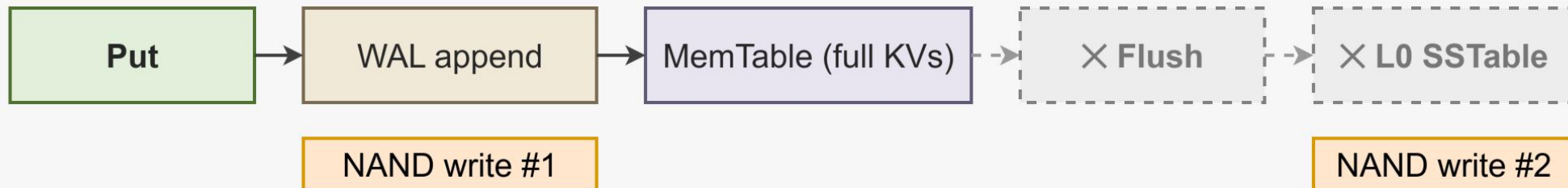
7. Design (2): WAL + Flush Folded into One Write

Fold WAL + flush into one write:

flush thread, flush I/O and the L0 tier are eliminated by construction → pre-L1 NAND passes 2× → 1×.

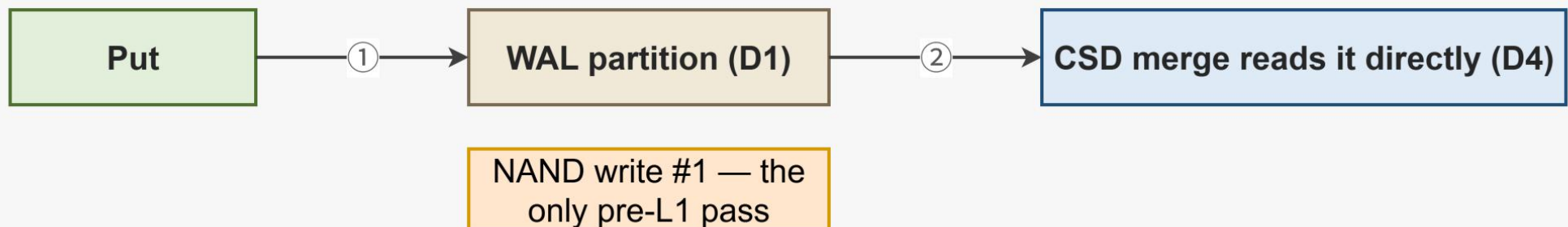
Maps to: Pain 1 (redundant pass removed) + Gap (eliminate, not accelerate).

Traditional path — two pre-L1 NAND passes



vs

ZeroFlush path — one pre-L1 NAND pass

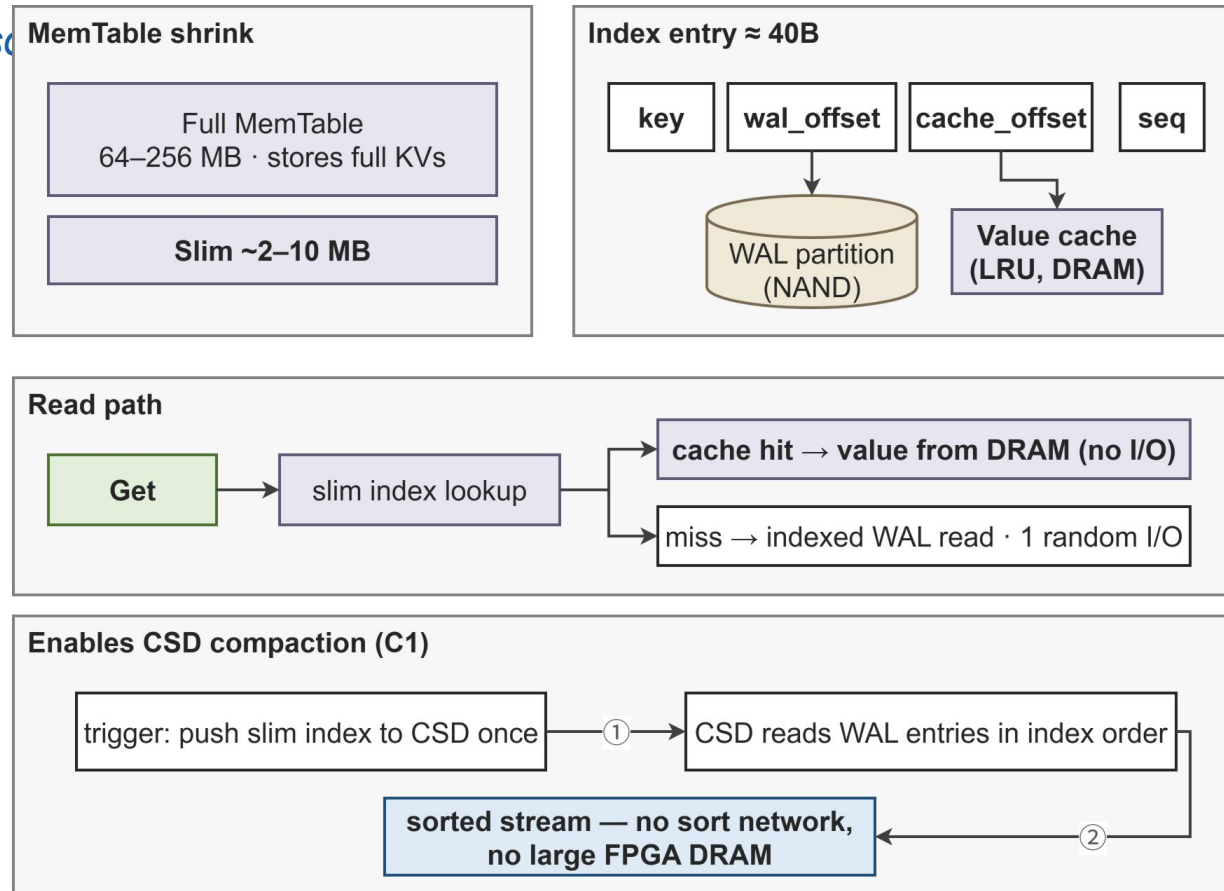


7. Design (3): Slim MemTable - Index + Value Cache

Slim MemTable = index + value cache:

entry $\approx 40\text{B}$ (key, wal_offset, cache_offset, seq); 64–256 MB $\rightarrow$ ~2–10 MB; index pushed once $\rightarrow$ CSD reads WAL in index order (sorted stream).

Maps to: C1 (index-guided scan)

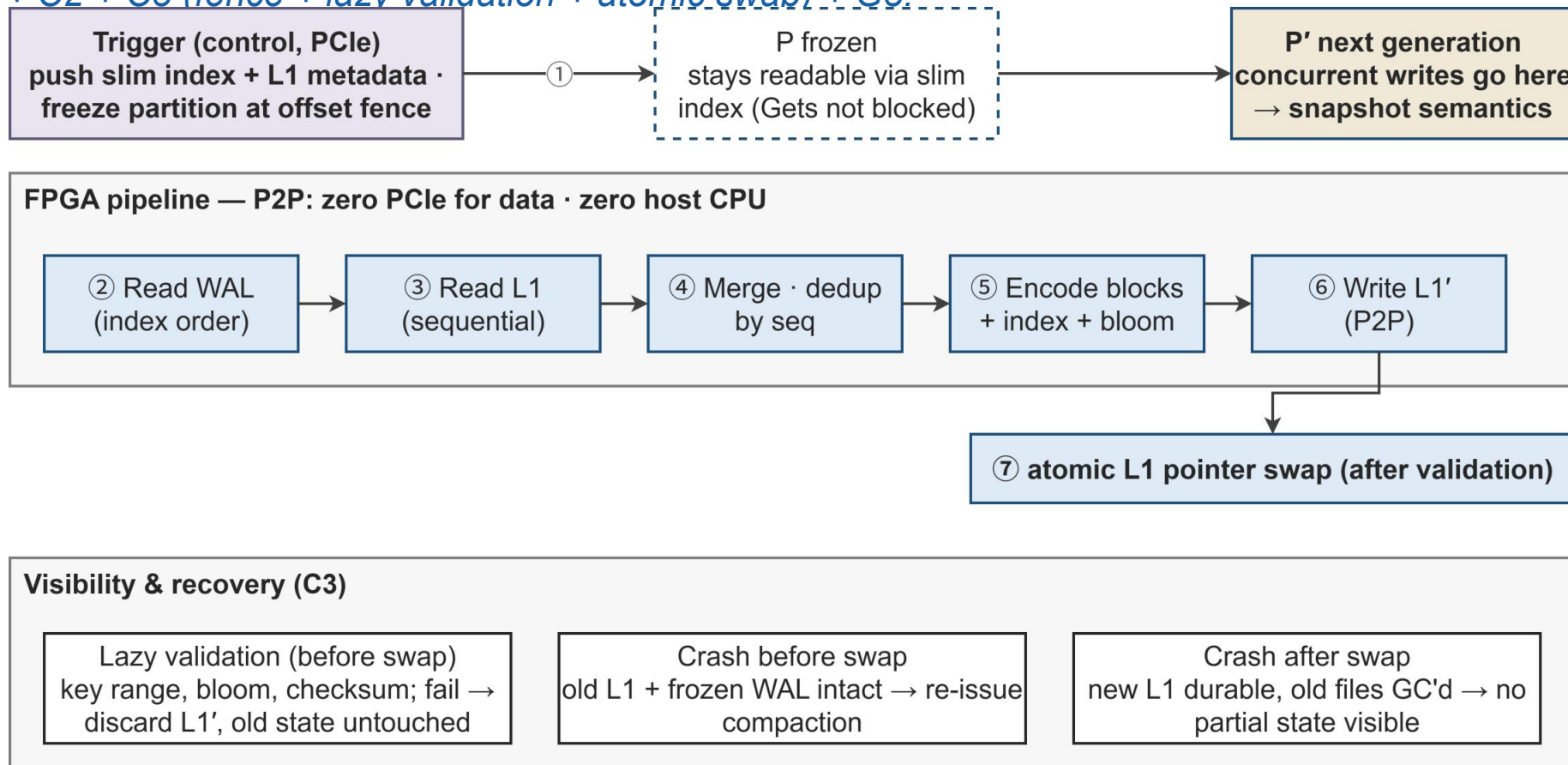


7. Design (4): CSD Merge Compaction

CSD merge compaction (FPGA):

freeze → 5-stage P2P pipeline (read WAL + L1 → merge → encode → write L1') → lazy validation → atomic pointer swap.

Maps to: C1 + C2 + C3 (fence + lazy validation + atomic swap) + G3.

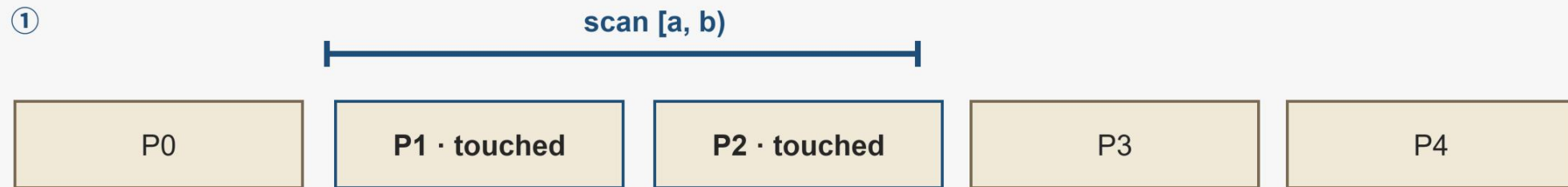


7. Design (5): Adaptive Range Scan

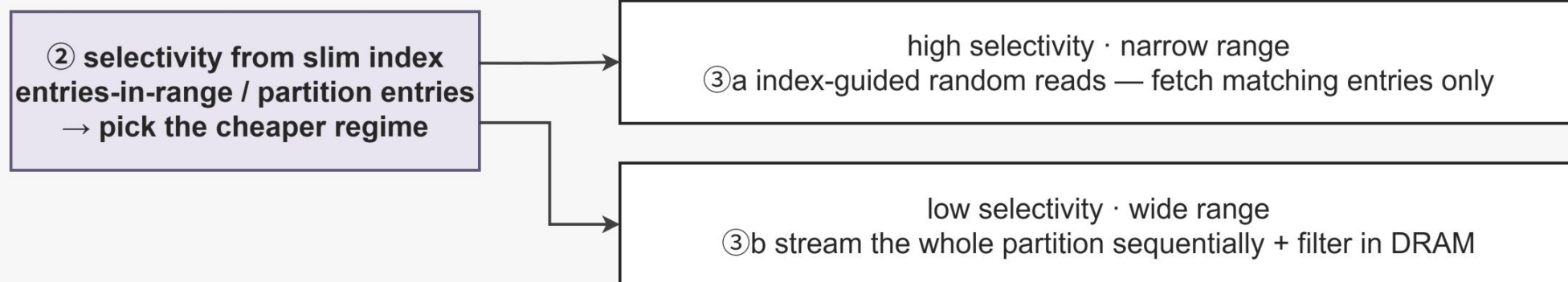
Adaptive range scan:

cross-partition: $q + n$ merge iterators vs $k + n$ L0 files ($q \ll k$); within partition: narrow $\rightarrow$ indexed random reads, wide $\rightarrow$ stream + DRAM filter.

Maps to Partition level — cross-partition is an advantage



Within a partition (arrival order) — the cost, absorbed by policy



CSD FPGA Scheduling: Control Plane + Execution Plane

Design choice: split scheduling by time scale.

1. Put global policy on the CSD CPU

Admission, priorities, reconfiguration and recovery are complex and change often.

2. Put the local fast path on the FPGA

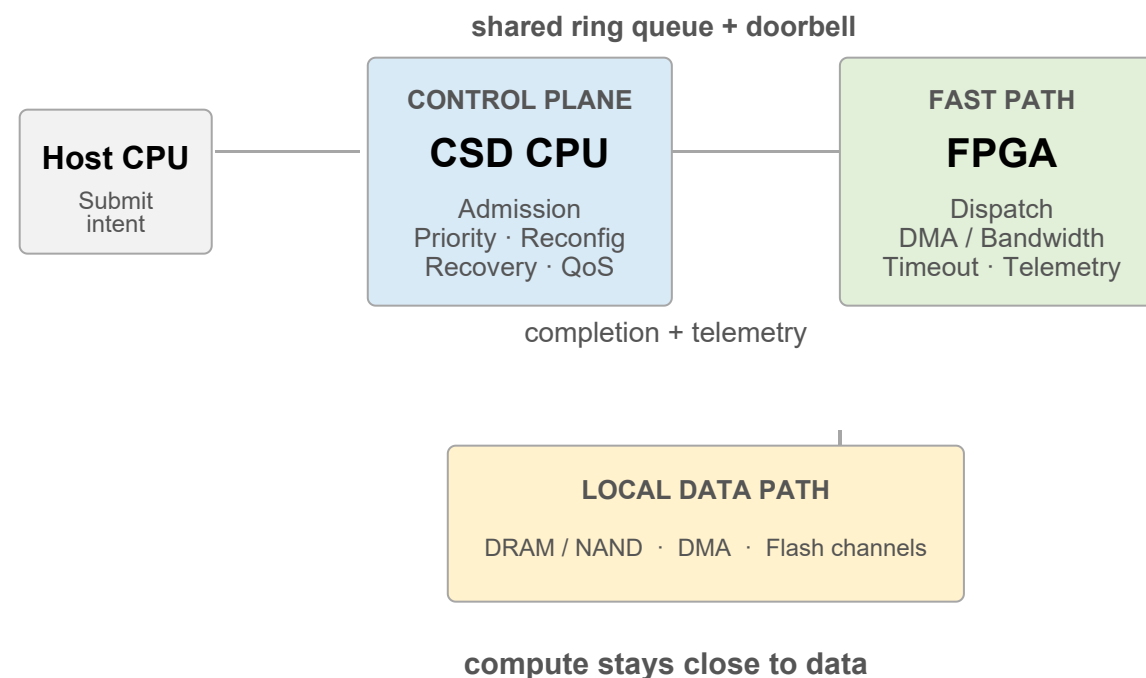
Dispatch, DMA and bandwidth arbitration are repetitive and latency-critical.

3. Connect both planes with shared queues

Ring queues, doorbells and completion records batch work and remove per-job host round trips.

4. Track the drain-rate envelope

CSD CPU monitors merge throughput vs ingest rate; admission backpressure is raised here.



8. Evaluation

Workload: db_bench; YCSB A-D, F; YCSB-E range scans + recency-skewed variant

Baselines: RocksDB (no CSD); AegonKV (CSD offload -> isolates eliminate vs accelerate)

Key experiments:

- pre-L1 WA and end-to-end WA (reported separately)
- read/write throughput, avg + tail latency; range-scan throughput
- sustained-write runs; total stall time; admission-backpressure share

Device-throughput checks: CSD merge/encode throughput vs ingest rate; routing cost

Ablation: D1-only (partitioning over stock flush) vs full ZeroFlush

Sensitivity: key skew, read/write ratio, value size, cache capacity

Correctness: crash injection + MVCC consistency checks

Host CPU utilization (offload efficacy)

Thank You